

Primitive number data types

INTRODUCTION TO JAVA



Jim White
Java Developer

Primitive types

	Primitive type	Value	Example
whole numbers	byte	-128 to 127	<code>byte age = 5;</code>
	short	-32,768 to 32,767	<code>short sqFeet = 3240;</code>
	int	-2,147,483 and 2,147,647	<code>int accountBalanace = 100000;</code>
	long	-2^63 and 2^63	<code>long nationalDebt = 34849081491943L;</code>
decimals	float	3.4e-038 to 3.4e+038	<code>float unitPrice = 4.99f</code>
	double	1.7e-308 to 1.7e+308	<code>double pi = 3.141592653589793;</code>
	char	alphabet, punc, numbers, etc.	<code>char grade = 'C';</code>
	boolean	true or false	<code>boolean isZoned = true;</code>

Numerical primitive types

	Primitive type	Value	Example
whole numbers	byte	-128 to 127	<code>byte age = 5;</code>
	short	-32,768 to 32,767	<code>short sqFeet = 3240;</code>
	int	-2,147,483 and 2,147,647	<code>int accountBalanace = 100000;</code>
	long	-2^63 and 2^63	<code>long nationalDebt = 34849081491943L;</code>
decimals	float	3.4e-038 to 3.4e+038	<code>float unitPrice = 4.99f</code>
	double	1.7e-308 to 1.7e+308	<code>double pi = 3.141592653589793;</code>
	char	alphabet, punc, numbers, etc.	<code>char grade = 'C';</code>
	boolean	true Or false	<code>boolean isZoned = true;</code>

Primitives for whole numbers - int

Type:	For:	Value:	Example:
-------	------	--------	----------

int	Used most often for whole numbers	Up to two million	467 902
-----	-----------------------------------	-------------------	---------

long, short, and byte

Type:	For:	Value:	Example:
long	Very large numbers	Up to quintillion	10 003 235
int	Used most often for whole numbers	Up to two million	467 902
short	Smaller numbers	Up to 30 thousand	15 899
byte	Very small numbers	-127 to 128	6

int is the primitive of choice

Type:	For:	Value:	Example:
long	Very large numbers	Up to quintillion	10 003 235
int	Used most often for whole numbers	Up to two million	467 902
short	Smaller numbers	Up to 30 thousand	15 899
byte	Very small numbers	-127 to 128	6

double

Type:	For:	Value:	Example:
<code>double</code>	Used most often for decimal numbers	Up to 16 decimal digits	3.1415926535

float

Type:	For:	Value:	Example:
double	Used most often for decimal numbers	Up to 16 decimal digits	3.1415926535
float	Smaller numbers, less precise	Up to 7 decimal digits	99.9

double is the primitive for decimal numbers

Type: For: Value: Example:

double

Used most often
for decimal
numbers

Up to 16 decimal digits

3.1415926535

float

Smaller numbers,
less precise

Up to 7 decimal digits

99.9

Declaring primitives - int

Syntax for:

`int`

```
int intEx = 123456;
```

Declaring primitives - int, double, short, byte

Syntax for:

int

double

short

byte

```
int intEx = 123456;
```

```
double doubleEx = 3.1415926535;
```

```
short shortEx = 394;
```

```
byte byteEx = 15;
```

Declaring primitives - float and long

Syntax for:

int

double

short

byte

float

long

```
int intEx = 123456;
```

```
double doubleEx = 3.1415926535;
```

```
short shortEx = 394;
```

```
byte byteEx = 15;
```

```
float floatEx1 = 99.9f;
```

```
float floatEx2 = 0.99F;
```

```
long longEx1 = 123456789901L;
```

```
long longEx2 = 987654321L;
```

Without F/f or L/l -> **error**

Can use either upper or lowercase

Numerical primitives in action

```
class PrintlnAsCalculator {  
    public static void main(String[] args) {  
  
        int x = 60000;  
        byte y = 1;  
        short z = 300;  
        long w = 200000000L;  
  
        System.out.println(x + y + z + w);  
    }  
}
```

20060301

```
class PrintlnAsCalculator {  
    public static void main(String[] args) {  
  
        double pi = 3.1415926535;  
        float half = 0.5F;  
  
        System.out.println(pi + half);  
    }  
}
```

3.6415926535

Let's practice!

INTRODUCTION TO JAVA

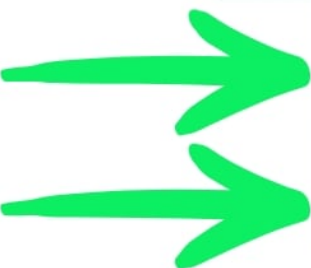
Other primitives and Strings

INTRODUCTION TO JAVA



Jim White
Java Developer

Primitives continued

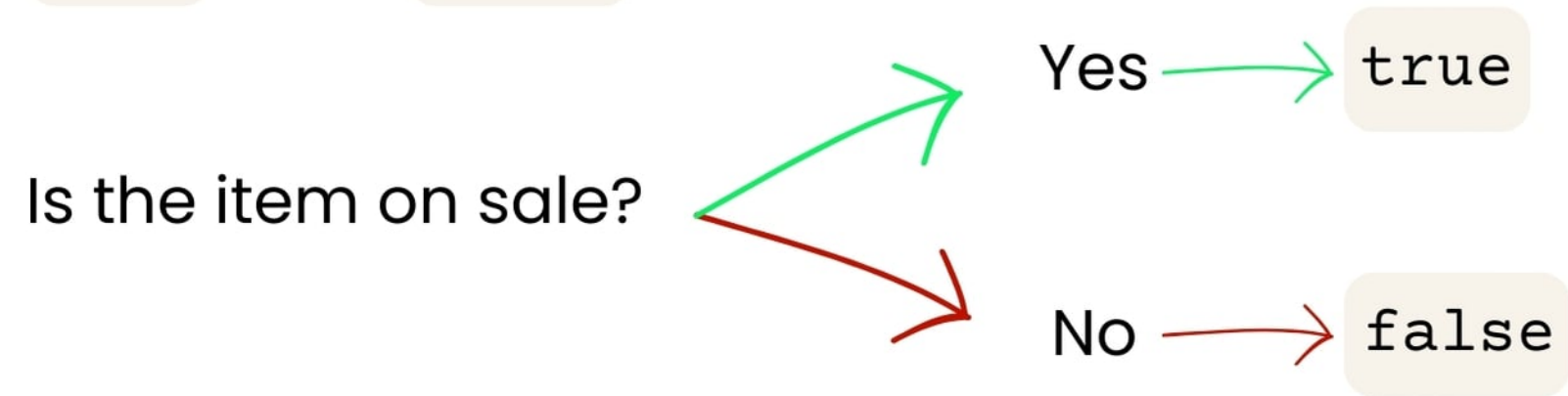
	Primitive type	Value	Example
whole numbers	byte	-128 to 127	<code>byte age = 5;</code>
	short	-32,768 to 32,767	<code>short sqFeet = 3240;</code>
	int	-2,147,483 and 2,147,647	<code>int accountBalanace = 100000;</code>
	long	-2^63 and 2^63	<code>long nationalDebt = 34849081491943L;</code>
decimals	float	3.4e-038 to 3.4e+038	<code>float unitPrice = 4.99f</code>
	double	1.7e-308 to 1.7e+308	<code>double pi = 3.141592653589793;</code>
	char	alphabet, punc, numbers, etc.	<code>char grade = 'C';</code>
	boolean	true or false	<code>boolean isZoned = true;</code>

booleans

true and false

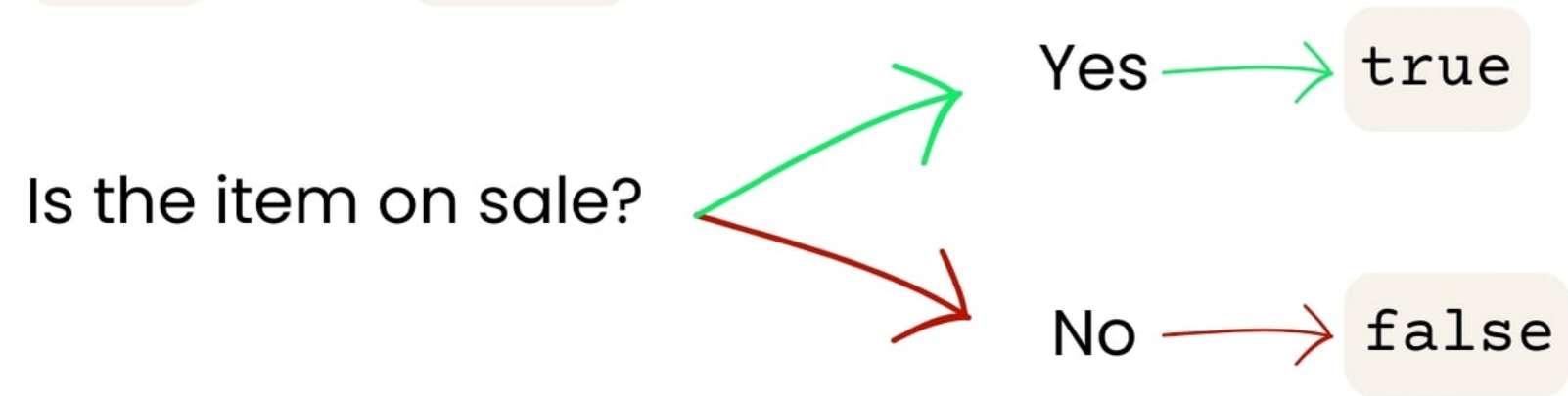
booleans - how they work?

`true` and `false`



booleans - naming convention

true and false



- Variable names start with `is`

```
boolean isOnSale = true;
```

```
boolean isFullPrice = false;
```

char

- Stores single character
- Wrapped in single quotes

```
// Examples
```

```
char initial = 'J';
```

```
char size = 'M';
```

```
char ampersand = '&';
```

char versus String

char

- Single quotes

```
char initial = 'J';
```

- Primitive data type
- Stores one character

String

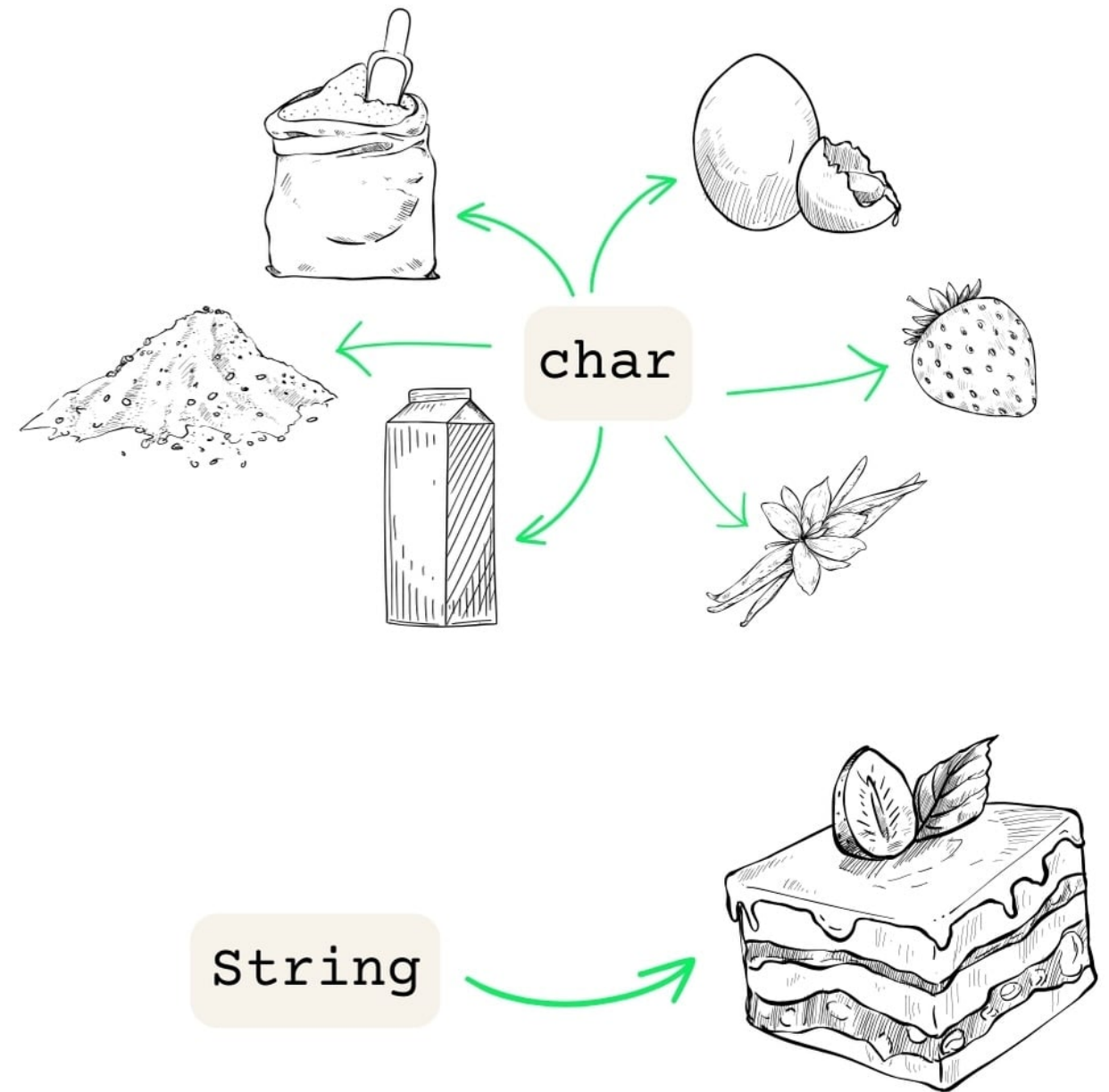
- Double quotes

```
String name = "Jim";
```

- Not a primitive, collection of characters
- Stores text

Strings

- Collection of characters inside double quotes
- Any number of characters
 - Single character in double quotes is `String`
 - Single character in single quotes is `char`
- Not primitives
 - Come with built-in functionality



Storing Strings

```
String variableName = "Text to store";
```

```
//Example
```

```
String myName = "Jim White";
```

Don't forget the double quotes!

Summary of primitives

	Primitive type	Value	Example
whole numbers	byte	-128 to 127	<code>byte age = 5;</code>
	short	-32,768 to 32,767	<code>short sqFeet = 3240;</code>
	int	-2,147,483 and 2,147,647	<code>int accountBalanace = 100000;</code>
	long	-2 ⁶³ and 2 ⁶³	<code>long nationalDebt = 34849081491943L;</code>
decimals	float	3.4e-038 to 3.4e+038	<code>float unitPrice = 4.99f</code>
	double	1.7e-308 to 1.7e+308	<code>double pi = 3.141592653589793;</code>
	char	alphabet, punc, numbers, etc.	<code>char grade = 'C';</code>
	boolean	true or false	<code>boolean isZoned = true;</code>

Used most often:

Summary of Strings

- Collection of characters (even single characters)
- Stores text
- Built-in functionality (more in next video)
- Use double quotes

```
// Example
```

```
String myName = "Jim White";
```

Let's practice!

INTRODUCTION TO JAVA

Working with Strings

INTRODUCTION TO JAVA



Jim White
Java Developer

String recap

- Collection of characters (even single characters)
- Stores text
- Built-in functionality (more now)
- Use double quotes

```
// Example
```

```
String myName = "Jim White";
```

...

String.length()

- Number of characters in the `String` = the length of the `String`
- `String.length()` returns `int` for number of characters

```
String userName = "JSmith13";
```

```
int userNameLength = userName.length(); // Value is 8
```

Changing case

`.toLowerCase()`

- Changes all letters to lowercase

```
class StringMethods1 {  
    public static void main (String[] args){  
  
        String yes = "Yes";  
        System.out.println(yes.toLowerCase());  
  
    }  
}
```

yes

`.toUpperCase()`

- Changes all letter to uppercase

```
class StringMethods2 {  
    public static void main (String[] args){  
  
        String yes = "Yes";  
        System.out.println(yes.toUpperCase());  
  
    }  
}
```

YES

String concatenation

- Use `+` to combine two `String` s together

```
class StringMethodsCont {  
    public static void main (String[] args){  
  
        String message1 = "Java is";  
        String message2 = "awesome";  
        // Use " " to nicely format the final message  
        System.out.println(message1 + " " + message2);  
    }  
}
```

Java is awesome

Concatenation with numbers

- `+` automatically converts numbers to `String` when used with a `String`

```
class StringMethodsCont {  
    public static void main (String[] args){  
  
        String message1 = "Java is";  
        int message2 = 29;  
        // Use " " to nicely format the final message  
        System.out.println(message1 + " " + message2);  
    }  
}
```

Java is 29

Declare now, assign later

- Declare variable without giving them value, assign the value later
- Useful for
 - Large amounts of text
 - When the value is not yet known
 - Effective structuring of code

```
// Define now
String courseCreators;

// Assign value later
courseCreators = "Instructor: Jim White; Collaborators: Kat, George, Arne, Eduardo";
```

Recap

- `String` = collection of characters in double quotes, with built-in functionality

```
// Declare now
```

```
String testMessage;
```

```
// Assign later
```

```
testMessage = "This is a test";
```

```
// Method to return the String's length (number of characters)
```

```
int msgLength = testMessage.length(); // Value is 14
```

```
// Methods to convert String to lower or upper case
```

```
String lowerMessage = testMessage.toLowerCase(); // Value is "this is a test"
```

```
String upperMessage = testMessage.toUpperCase(); // Value is "THIS IS A TEST"
```

Let's practice!

INTRODUCTION TO JAVA

Introduction to arrays

INTRODUCTION TO JAVA



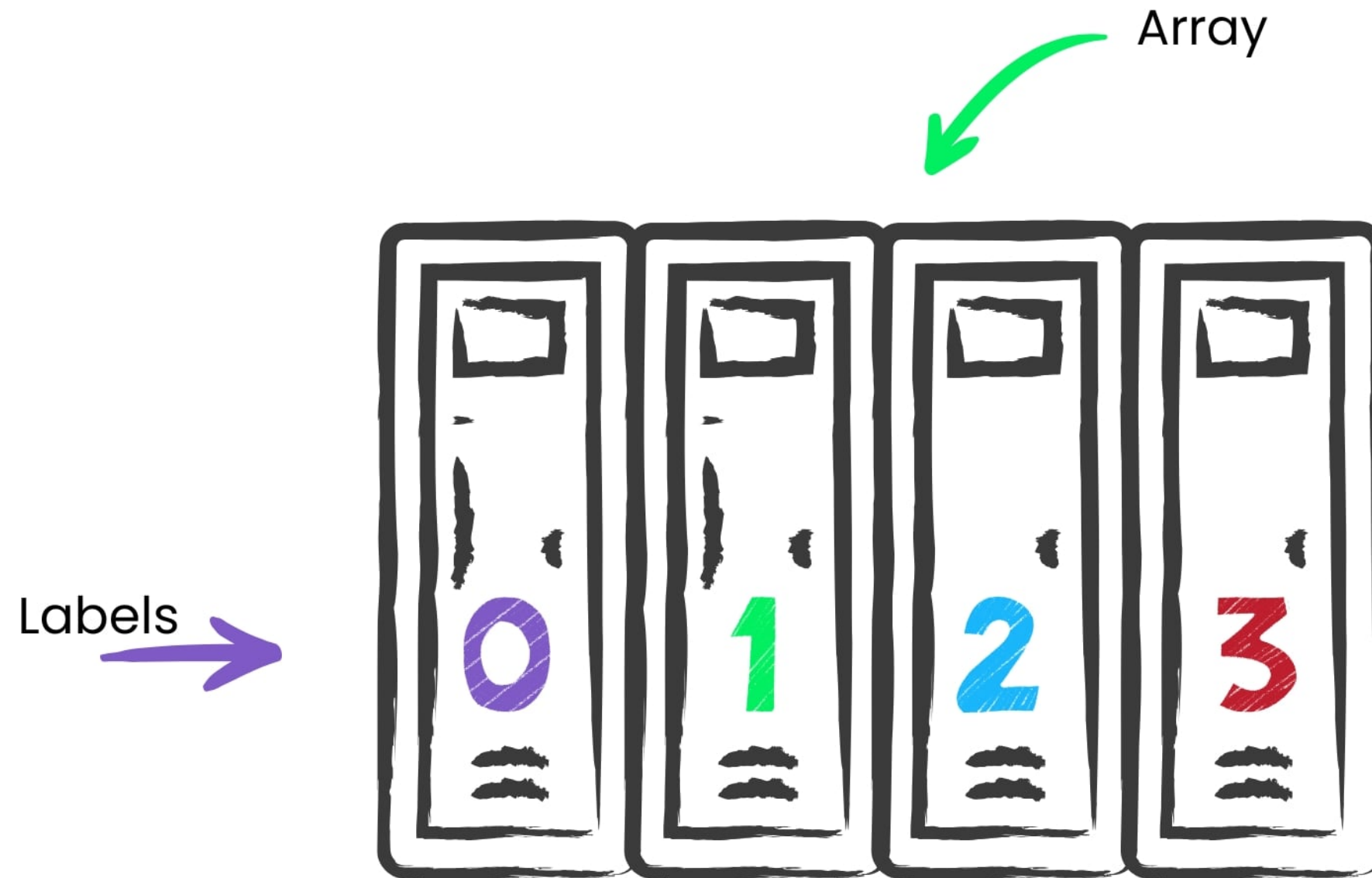
Jim White
Java Developer

Array is ...

- Collection of values stored as a single variable

Array is ... a row of lockers!

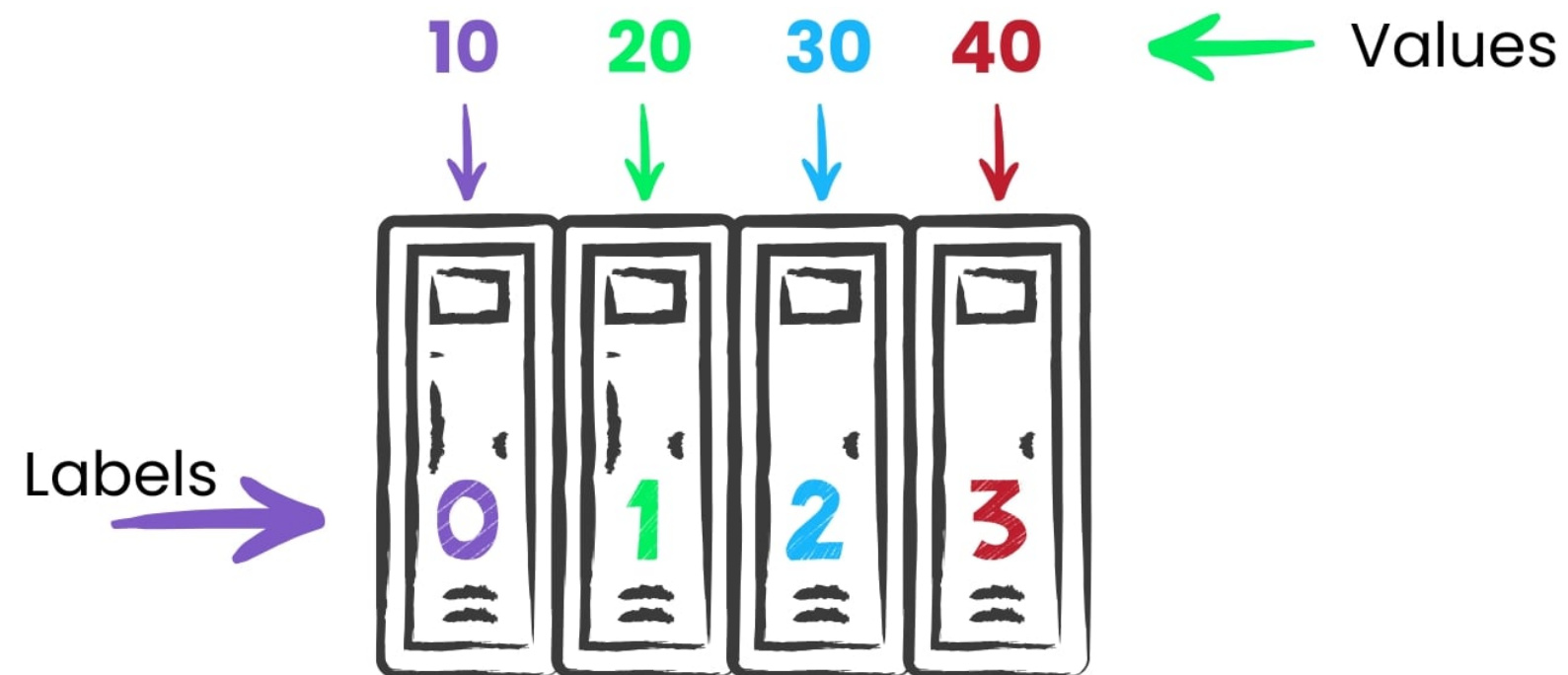
- Collection of values stored as a single variable



Declaring and filling arrays

- Using square brackets `[]`

```
// Declare array variable  
int[] prices;  
// Assign value  
prices = {10, 20, 30, 40};
```



Accessing elements

- Using `[]`:

```
int[] prices = {10, 20, 30, 40};  
  
// Accessing first element  
int firstElement = prices[0]; // Value is 10  
  
// Accessing second element  
int secondElement = prices[1]; // Value is 20
```

Java starts indexing from 0 !

Changing value of an element

- Change value by assigning new value to specific index

```
int[] prices = {10, 20, 30, 40};  
  
// Change value of third element to 95  
prices[2] = 95;
```



Array length is fixed

- Array's length is fixed on creation

```
int[] itemIDs = {10, 20, 30, 40, 50};  
  
// Accessing sixth element that isn't there  
itemIDs[5] = 60; // <- This will cause an error
```

- Check array's length using `.length` property

```
int[] prices = {10, 20, 30, 40};  
  
int pricesLength = prices.length; // Value is 4
```

Note – here, `.length` doesn't have any brackets!

Printing out values

We can print out individual values:

```
class ArrayElementPrinting {  
    public static void main (String[] args){  
        int[] prices = {10, 20, 30, 40};  
  
        // Printing element by element  
        System.out.println(prices[0]);  
        System.out.println(prices[1]);  
    }  
}
```

10

20

Trying to print an entire array

Printing whole array returns representation:

```
class ArrayPrinting {  
    public static void main (String[] args){  
        int[] prices = {10, 20, 30, 40};  
  
        // Printing whole thing  
        System.out.println(prices);  
    }  
}
```

```
[I@d041cf
```

Arrays of other things

```
// Array of Strings  
String[] productNames = {"Organic Honey",  
                           "Cold Brew Coffee",  
                           "Dark Chocolate Bar"};
```

Values in an array

- Not limited to just numbers or `Strings`

! All values in an array must have the same type !

Recap

- Arrays are useful of storing multiple values of the **same type**
- Arrays are indexed from 0
- Arrays have fixed length

```
// Declare and assign
int[] prices = {10, 20, 30, 40};

// Retrieve
int secondElement = prices[1];

// Update
prices[1] = 25;

// Length
// Value is 4
int pricesLength = prices.length;
```


Let's practice!

INTRODUCTION TO JAVA